

Sora

Architecture and design of Portage's built-in AI assistant

Local model · Propose, then approve · Redaction on by default

Portage v0.1 · Document date: 9 October 2026
Audience: engineers, product owners, security reviewers
Status: Phases 0-2 delivered, read-only operations tools included

0 Contents

1 Overview and goals
2 Decisions at a glance
3 System architecture
4 Request lifecycle
5 Propose, then approve
6 Skills
7 Tool catalog
8 Mapping versions and naming
9 Safety and security model
10 Knowledge
11 Data model
12 API reference
13 Frontend experience
14 Configuration
15 Testing and validation
16 Operations and troubleshooting
17 Limitations and roadmap

1 Overview and goals

Sora is an assistant built into Portage. Users type requests in plain English, such as "*map Rally Iteration to the Jira sprint field for User Story and save it as jira_us_mapping_v5*" or "*add Sprint 24 with Jira ID 1042 to the sprint catalog*", and Sora carries them out inside the product. It is comparable in spirit to Jira's Rovo: context-aware chat, specialised skills, grounded answers, and actions that stay under human control.

Local
Runs on Ollama on the same machine. No data leaves it.

Approved
Every change is a proposal the user applies.

Reversible
New mapping versions; versioned catalogs with restore.

Traceable
Every chat, tool call and apply is audited.

What users can do today: edit and version mappings, add, change, remove and restore catalog values (sprint, version, user, team), find values in an upload that are missing from a catalog, ask why a migration job failed, look up connection status, and ask how Portage works.

What Sora does not do yet: start a migration, run attachment or description jobs, delete mappings or catalogs, or change connections and secrets.

2 Decisions at a glance

Topic	Decision	Why
Model	Local Ollama, default `llama3.1:8b`, native tool calling, streaming.	Self-hosted product; migration data stays inside the customer's network.
Privacy	Redaction of emails and secrets on by default; toggle in settings.	Catalogs contain user emails; defence in depth even for a local model.
Change control	Mode `propose` (default) or `readonly`.	A small model can misread a request; a human click is the safety net.
Runtime	Python 3.12 (matches the Docker image); the cryptography version cap was removed.	OpenSSL 3 build of Python; modern dependencies.
Orchestration	A small purpose-built loop (no agent framework).	Few dependencies, easy to test with a scripted model.
Versioning	Mappings reuse the Studio's name-based versions; catalogs get an `entity_versions` history.	Studio already never overwrites a mapping; catalogs were overwritten in place.

3 System architecture

The browser sends the user's message and the current page context to the backend. The backend selects a skill, masks sensitive text, and streams a conversation with the local model. When the model asks for a tool, the backend validates and runs it; read tools return data, write tools return a proposal. Everything is pushed to the browser as server-sent events.

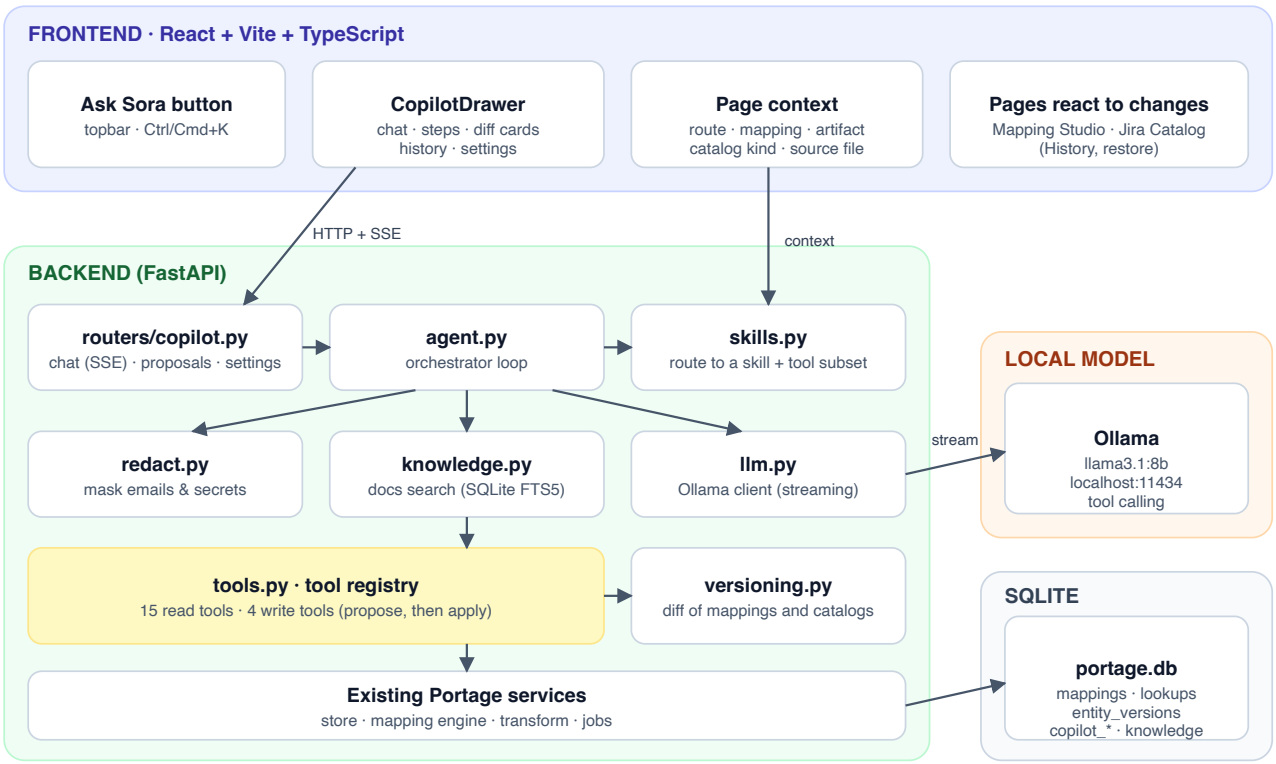


Figure 1. Components and data flow

Backend modules (`backend/app/copilot` and `routers/copilot.py`)

Module	Responsibility
`agent.py`	Builds the system prompt (rules, skill, page context), keeps the last 10 messages, runs the stream-and-tool loop (max steps, default 6), masks outgoing text and restores incoming text, persists messages, emits events.
`skills.py`	Skill definitions, tool subsets, routing by @mention, keywords and page route, per-page suggestions.
`tools.py`	Typed tool registry. Validates arguments (Pydantic), tolerates small-model quirks (nested arguments sent as JSON strings, "null" for optional values), creates proposals, applies them.
`llm.py`	Ollama client: streams `/api/chat` with tools, temperature 0.1, 8,192-token context; health check; friendly errors when Ollama or the model is missing.
`redact.py`	Reversible email pseudonymisation, one-way secret masking, and a streaming restorer that handles placeholders split across tokens.
`knowledge.py`	Builds an FTS5 index from a built-in glossary plus README.md and PRODUCT.md and searches it with BM25.
`settings.py`	Sora configuration stored in `app_settings` , overridable by environment variables.
`versioning.py`	Pure diff functions for catalogs and mapping templates.
`routers/copilot.py`	HTTP API: chat stream, proposals, conversations, settings, status, audit.

4 Request lifecycle

One user turn, from message to applied change. The model never writes data: steps 5 to 7 create a pending proposal, and only the user's click in step 12 triggers the write in step 13.

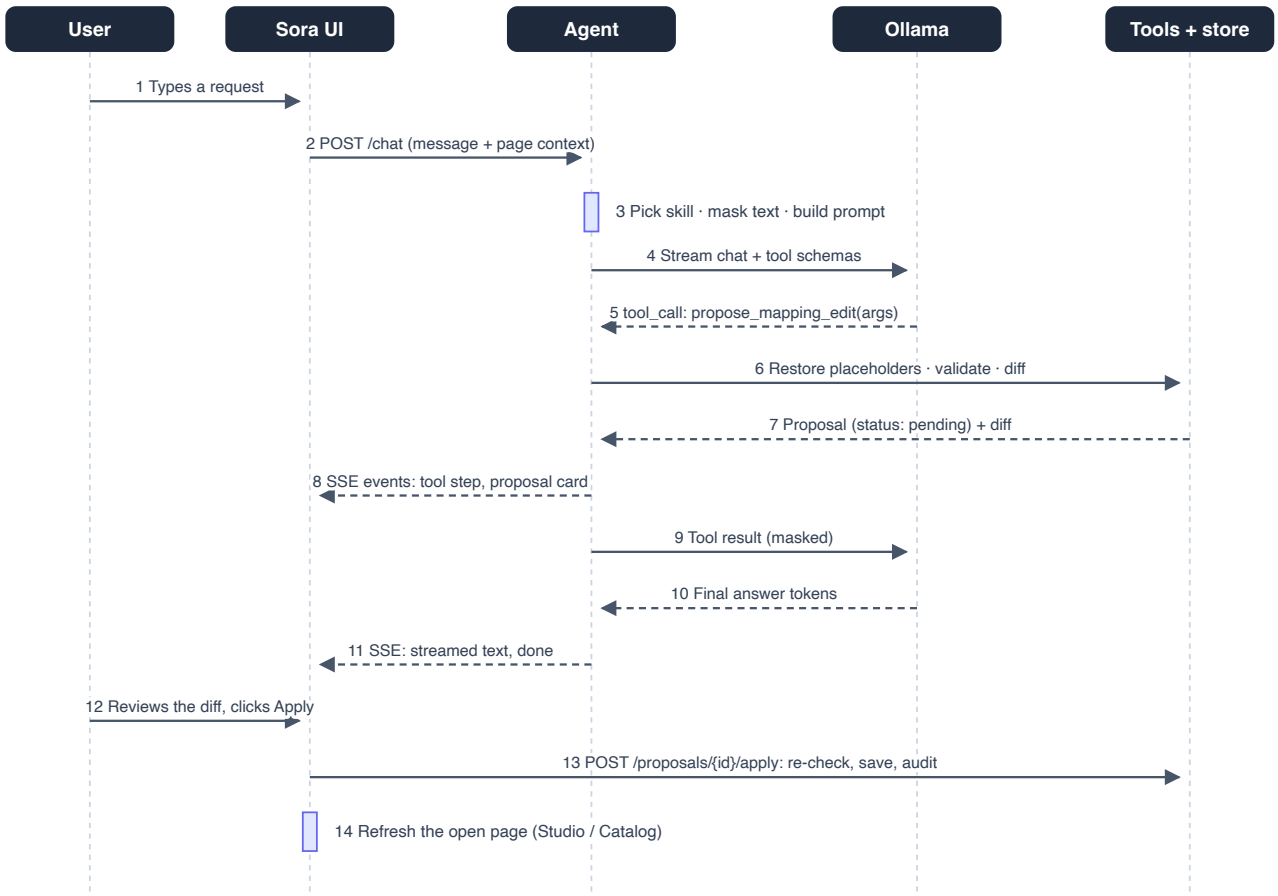


Figure 2. Sequence of a mapping edit (dashed arrows are responses)

5 Propose, then approve

Model asks calls propose_* with arguments → **Validate** names, fields, versions; clear error if invalid → **Diff** compare with current state → **Proposal** stored as pending, shown as a card → **User decides** Apply or Reject → **Apply** re-derive, re-check, write, audit

Validation errors are returned to the model as tool results so it can correct itself (for example, a version name that already exists comes back with the next free name). A proposal that has no effect, such as setting a mapping that already exists, is refused instead of creating an empty version.

> **Server-enforced.** The model cannot skip approval: write tools have no code path that writes. Applying is a separate API call, blocked in read-only mode, and refused (409) if the proposal was already applied or rejected.

6 Skills

A skill is a focused persona with its own instructions and a restricted tool set, which keeps a small model accurate. Routing uses an explicit @mention, then keyword and page-route scoring, and falls back to the general skill.

Skill	Mention	Default pages	Purpose	Tools
Mapping Architect	`@mapping`	/mapping	Edit field mappings and save new versions.	list_mappings, get_mapping, compare_mappings, list_mapping_fields, propose_mapping_edit, list_sources, describe_source, search_docs
Catalog Curator	`@catalog`	/lookups	Maintain sprint, version, user and team catalogs.	catalog read tools, find_unmapped_values, distinct_values, list_sources, describe_source, propose_catalog_*, search_docs
Migration Operator	`@migration`	/migrate, /reports, /exports, /attachments	Inspect jobs, failures and readiness. Cannot start a migration.	list_jobs, get_job, list_sources, describe_source, find_unmapped_values, list_mappings, connection_status, search_docs
Guide	`@guide`	/connections	Explain how Portage works from the documentation.	search_docs, connection_status, list_mappings, list_catalogs
Sora (general)	`(none)`	any	Fallback when no skill matches. Can use every tool.	all

7 Tool catalog

15 read tools (**read**) run immediately. 4 write tools (**write**) only create proposals. Tool schemas are generated from Pydantic models and flattened for the model.

Tool	Risk	What it does
`list_mappings`	**read**	Saved mapping versions with id, name and field counts per artifact.
`get_mapping`	**read**	Field maps of one mapping, optionally for one artifact; value-map summary.
`compare_mappings`	**read**	Diff two saved mapping versions (added, removed, changed per artifact).
`list_mapping_fields`	**read**	Valid Rally source columns and Jira target fields for an artifact.
`list_catalogs`	**read**	Sprint, version, user and team catalogs with entry counts.
`get_catalog_entries`	**read**	Search entries of a catalog by text.
`list_catalog_versions`	**read**	Version history of a catalog.
`list_sources`	**read**	Recently uploaded source CSV files.
`describe_source`	**read**	Columns, row count and sample rows of an upload (defaults to the open or latest file).
`distinct_values`	**read**	Distinct values and counts for a column of an upload.
`find_unmapped_values`	**read**	Values in an upload that are missing from the sprint/version/user/team catalog.
`list_jobs / get_job`	**read**	Recent migration jobs; results, failed items and log tail of one job.
`search_docs`	**read**	Keyword search over Portage documentation and built-in concepts.
`connection_status`	**read**	Whether Jira/Rally connections are saved. Never returns secrets.
`propose_mapping_edit`	**write**	Set or remove field mappings for one artifact and save as a new mapping version, with an optional user-chosen version name.
`propose_catalog_upsert`	**write**	Add or update entries in a catalog (versioned).
`propose_catalog_remove`	**write**	Remove entries from a catalog (versioned).
`propose_catalog_restore`	**write**	Restore a catalog to an earlier version (undo).

8 Mapping versions and naming

The Mapping Studio never overwrites a mapping: each save is a new row named `jira_<artifact>_mapping_vN`. Sora follows the same convention and lets the user choose the name in chat.

User says	Saved as
"... save it as `jira_us_mapping_v5`"	Exactly that name; version number 5.
"... call it v6" for Defect	`jira_defect_mapping_v6` (shorthand v6, V6 or 6 expands per artifact).
"... call it Q4 sprint fix"	That free-form name; version number is the next one.
(no name)	Next number for the artifact, e.g. `jira_us_mapping_v3`.
A name that already exists	Refused with the next free name; checked again at Apply in case it was taken meanwhile.
A name for a different artifact, or with unsafe characters or over 80 characters	Refused with an explanation.

Edit operations. `set` maps one or more Rally source columns (the first non-empty wins) to a Jira target; `remove` deletes a mapping. Field and column names are resolved case-insensitively against the field catalog. The transform follows the target's data type (array to labels, number, date). Other artifacts and value maps are left untouched, and older versions are never modified.

9 Safety and security model

Risk	Control
The model changes data without consent	Write tools only create proposals. A separate Apply endpoint, called by the user's click, performs the write. Enforced on the server, not by the prompt.
A proposal is stale when applied	Apply re-derives the change from the stored arguments against current data, re-checks the version name, and returns 409 if the proposal was already applied or rejected.
Personal data reaches the model	Emails become stable placeholders (<email_1>) and are restored in tool arguments and streamed answers. Tokens and key=value secrets are masked one-way. On by default; the model runs locally anyway.
Credentials exposed	No tool reads Jira or Rally tokens. connection_status returns only booleans, base URL and project key. No tool can set a secret.
Prompt injection from data	Tool output is passed back as data. No tool sends data to external systems, and delete or run-migration tools are not exposed. Every change still needs a human click.
Path traversal via source ids	A source id must be 32 hex characters, otherwise it is ignored and the current or latest upload is used. It is never treated as a path.
Unsafe or colliding version names	Names allow letters, digits, space, _ - . up to 80 characters; a name for another artifact is refused; duplicates (case-insensitive) are refused with the next free name.
Hallucinated or disallowed tools	Calls are resolved through the registry and the active skill and mode. Unknown or unavailable tools return an error. Read-only mode removes write tools.
Runaway loops and oversized context	Max 6 steps per turn (configurable 1-12), tool results truncated to 6,000 characters, last 10 messages of history.
Traceability and undo	copilot_audit records chats, tool calls, proposals and applies. Catalog changes are versioned with author = copilot; mapping versions are immutable rows.

> **Known gap.** Portage has no user authentication yet, so mode and permissions are global rather than per user. Person names inside catalogs are not detected by the redactor.

10 Knowledge

- **Documentation:** a built-in glossary (Mapping Studio, Jira Catalog, value maps, migration, connections, Sora) plus README.md and PRODUCT.md, chunked by heading and searched with SQLite FTS5 (BM25). No embeddings or external services are needed.
- **Live workspace data:** not embedded. It is read through tools at question time, so answers reflect current mappings, catalogs, uploads and jobs.
- **Page context:** the route, selected mapping, artifact, catalog kind and open source file are sent with each message and added to the prompt.

11 Data model

Table	Purpose
`entity_versions`	Catalog history: entity_type, entity_id, version, payload, author (user | copilot), message, created_at. Unique per entity and version; identical snapshots are skipped; the pre-versioning state is stored as a baseline on first update.
`copilot_conversations / copilot_messages`	Chat history shown in the History view (role, content, metadata such as skill, tools and proposal ids).
`copilot_proposals`	Proposal id, conversation, tool, risk, arguments, preview diff, status (pending, applied, rejected), result.
`copilot_audit`	Append-only event log: chat, tool, proposal_created, proposal_applied, proposal_rejected, mapping_saved, catalog_saved, llm_error.
`knowledge (FTS5)`	Chunks of the built-in glossary, README.md and PRODUCT.md, rebuilt at startup, searched with BM25.
`app_settings`	Key copilot_settings holds the Sora configuration as JSON (no secrets).
`mappings`	Unchanged. Each Studio or Sora save is a new row named jira_<artifact>_mapping_vN.

12 API reference

Method	Path	Description
POST	`/api/copilot/chat`	Stream a turn as server-sent events: meta, token, tool_call, tool_result, proposal, error, done.
POST	`/api/copilot/proposals/{id}/apply`	Apply a pending proposal (403 in read-only, 409 if already resolved, 422 on a validation failure).
POST	`/api/copilot/proposals/{id}/reject`	Reject a pending proposal.
GET	`/api/copilot/status`	Settings plus Ollama reachability and whether the model is installed.
GET / PUT	`/api/copilot/settings`	Read or update enabled, base_url, model, mode, redact, max_steps.
GET	`/api/copilot/skills, /suggestions?route=`	Skill list and per-page suggested prompts.
GET / DELETE	`/api/copilot/conversations[/{id}]`	Conversation list, one conversation with its proposals, delete.
GET	`/api/copilot/audit`	Recent audit events.
GET	`/api/lookups/{id}/versions, /diff`	Catalog version list and diff against the current version.
POST	`/api/lookups/{id}/restore/{version}`	Restore a catalog version (creates a new version).
GET	`/api/mappings/{id}/diff/{other}`	Diff two saved mapping versions.

Chat stream events

```
data: {"type":"meta","conversation_id":"...","skill":{"key":"mapping","label":"Mapping Architect"}}
data: {"type":"tool_call","name":"propose_mapping_edit","args":{...}}
data: {"type":"tool_result","name":"propose_mapping_edit","ok":true,"summary":"1 change(s)"}
data: {"type":"proposal","proposal":{"id":"...","status":"pending","preview":{"new_name":"jira_us_mapping_v5","diff":
{...}}}}
data: {"type":"token","text":"Proposed ..."}
data: {"type":"done","conversation_id":"..."}
```

13 Frontend experience

- **Entry points:** the *Ask Sora* button in the topbar and Ctrl/Cmd+K; Esc closes the panel.
- **Drawer:** streamed answers with light formatting, collapsible tool steps ("Looked up mapping..."), diff cards with Apply and Reject, suggested prompts per page, conversation history, and a settings panel (Ollama URL, model, mode, redaction) with a live connection check.
- **Context:** Mapping Studio and Jira Catalog tell Sora what is selected through `useCopilotPage`.
- **Live refresh:** after Apply, a `portage:data-changed` event reloads the Catalog page, or lists the new version in the Studio dropdown without disturbing the canvas.
- **Catalog History:** a modal on the Jira Catalog page lists versions with author (user or copilot), compares any version to the current one, and restores it.
- **Identity:** a gradient tile with a flowing "S" ribbon and a spark (`frontend/public/sora.svg` and the `SoraIcon` component).

14 Configuration

Setting	Environment variable	Default and notes
`base_url`	`PORTAGE_OLLAMA_URL`	http://localhost:11434
`model`	`PORTAGE_COPILOT_MODEL`	llama3.1:8b (must support tool calling)
`mode`	`PORTAGE_COPILOT_MODE`	propose; or readonly (no write tools, Apply refused)
`redact`	settings panel	true
`enabled`	settings panel	true
`max_steps`	settings panel	6 (allowed 1-12)

Settings saved in the panel are stored in the database and take precedence over the environment defaults.

15 Testing and validation

78
offline tests, all passing

5
opt-in tests against the real model

4
deliberate code breaks, all caught

0.8 s
offline suite run time

File	Tests	Covers
`test_copilot.py`	13	Redaction round trip and streaming, catalog versions and baseline, proposal then apply, restore, agent loop with a scripted model, apply and read-only endpoints, docs search, tool schemas.
`test_mapping_versions.py`	56	Naming rules (auto, full, shorthand, free-form, unsafe, duplicate, wrong artifact, race at apply), edit semantics and validation, immutability, API round trip, code generation, real rows through the transform engine, migration preview and export, catalog lookups on top.

File	Tests	Covers
`test_chat_mapping_versions.py`	9	A name given in chat reaches the saved version, auto-numbering, recovery after a collision, invalid names, string and null arguments, skill and context, read-only protection.
`test_live_ollama.py`	5 (opt-in)	Against the real llama3.1:8b model with PORTAGE_LIVE_LLM=1: named, shorthand, free-form and unnamed requests, and duplicate-name safety.

Mutation check. The naming logic was broken four ways (ignore the requested name, skip the duplicate check, overwrite instead of creating a new row, skip the re-check at Apply). Each break made tests fail, and everything passed again after restoring the code.

Real-model findings. The 8B model sometimes sends nested arguments as JSON strings or writes "null" for omitted fields (now tolerated), and once stopped after a lookup without proposing the edit (fixed by prompt wording). Because small models are non-deterministic, the live tests allow up to three attempts per scenario, while the safety properties must hold every time.

```
cd backend && pip install pytest httpx && pytest tests
PORTAGE_LIVE_LLM=1 pytest tests/test_live_ollama.py     # against local Ollama
```

16 Operations and troubleshooting

Symptom	Check
"Local model unavailable" in the panel	Start Ollama (`ollama serve` or the menu-bar app) and run `ollama pull llama3.1:8b` . The settings panel shows reachability and model status; so does `GET /api/copilot/status` .
Where are the Ollama logs?	macOS app: `~/.ollama/logs/server.log` (follow with `tail -f`). For more detail, quit the app and run `OLLAMA_DEBUG=1 ollama serve` .
What did Sora do?	`GET /api/copilot/audit` , and the uvicorn terminal showing each chat request.
Page stuck on "Loading fields..."	The backend is not running or the Vite proxy cannot reach port 8000. Start it from `backend/` : `uvicorn app.main:app --reload --port 8000` .
Wrong database	The database lives in `./data` relative to where uvicorn starts; use `PORTAGE_DATA_DIR` to pin it.

17 Limitations and roadmap

Current limitations

- An 8B local model can misread complex or multi-step requests; the approval step is the safety net. A larger local model improves reliability.
- No per-user roles; mode and permissions are global.
- Names of people are not redacted; only emails and secrets are.
- One primary catalog per kind (the most recently updated); documentation search is keyword-based.
- No tools for running migrations, attachment or description jobs, or deleting data.

Next steps

Phase	Scope
Operations	Start a migration from chat with a dry run and explicit confirmation; attachments and copy-description jobs; proactive suggestions such as "47 sprints in this file are unmapped".
Control	Viewer, Editor and Admin roles; delete tools behind extra confirmation; per-tool approval policies.
Quality	Evaluation suite of golden prompts in CI; telemetry of tool success and proposal acceptance; prompt versioning.
Reach	Additional model providers behind the same interface; an MCP server exposing the tool registry so other agents can drive Portage.